TECHNISCHE UNIVERSITÄT
CHEMNITZ

Chemnitz Cultural Sites App

Datenbanken und Web-Techniken

Project Task

Submitted by: Juho Lee
Study Course: M.Sc. Web Engineering
Student ID: 855888

Index

Index 1
1. Features Overview 2
1.1 Required Features Implementation 2
1.2 Additional Features (Interactive Parts) 2
2. Architecture and Components 3
2.1 Frontend Components (User Interface based description) 3
2.2 Database Structure (MongoDB collections) 4
2.3 Backend Components (API & Logic) 4
3. Utilized Technologies 6
3.1 Frontend 6
3.2 Backend 6
3.3 Database 6
3.4 Technology References 7
5. Appendix 9
5.1 Figures 9
5.2 Web Service API Documentation 19

1. Features Overview

1.1 Required Features Implementation

This app meets all five of the following requirements from the task description using **MERN** stack technologies: it provides cultural sites both on a map and as a text-based result list, allows filtering and searching by keywords via a filter panel rendered on both the map and list pages, displays detailed information for each cultural site based on its category, enables users to register using Google OAuth2.0 and manage personal details like their name and bio, and allows registered users to add and remove favorite cultural sites from the given datasets.

1.2 Additional Features (Interactive Parts)

Beyond meeting those five core requirements above, the application also incorporates several interactive features.

1. Users can actively engage with cultural sites by **writing, modifying, and deleting reviews and ratings**.
2. The user **can propose a new place** with the right-click context menu on the map or **suggest updates or deletion of existing places** through side panel. These submissions are then handled by administrators, who can **view, review, and accept or reject proposals**, directly applying approved changes to the database. Administrators also have a right to **directly add, modify or delete any cultural site**.
3. From a security and management standpoint, the app implements **Google OAuth 2.0 authentication** utilizing JWTs secured in HTTP-only cookies, along with **role-based access control** that clearly distinguishes between ‘logged in’ status as well as ‘admin’ and ‘user’ permissions. This hierarchy extends to **admin user management**, allowing administrators to see user details and modify their role.

---

2. Architecture and Components

2.1 Frontend Components (User Interface based description)

There’s a provided layout (Figure 2), with a fixed header and footer persisting across all pages. The app follows the layout (Figure 3). The app has a responsive design that supports viewport widths from 320px, and has been checked to work on Chromium-based browsers and MS Edge browsers. Users sign in via the account manager panel (top-right of homepage, Figure 5), utilizing Google OAuth 2.0. Authentication relies on JWTs managed through HTTP-only cookies.

The interactive map Interface, built with react-leaflet, features category-dependent marker visualization, marker clustering, a filter panel, restricted Chemnitz bounds, and a current location button.

Right clicking the map brings up a context menu button to query nearby OSM data via Overpass API. This button query with extendedCulturalSiteQuery (/backend/config/osmData.js), enabling users to propose or admins to add new OSM locations that baseCulturalSiteQuery missed it (Figure 18, Figure 19, Figure 20). Since the search radius is set relatively narrowly to 50m, its normal operation can be checked by querying from the ‘Altes Heizhaus’ at TU Chemnitz or ‘St. Petri Kirche’ at Theaterplatz.

Clicking a map marker displays detailed cultural site information in the resizable Side Panel (Figure 4). It includes favorite (add/remove) and review (read, upload, update) functionalities. Role-based buttons ('Suggest an edit' for 'user'; 'edit' and 'delete' for 'admin') allow updating or deleting cultural sites (Figure 21).

The fixed Header (top of viewport) provides navigation button on the left (list/map toggle), a central homepage button with title, and an account manager button on the right (sign out, my-account).

Accessible via the header's left button, the List Page displays cultural sites in a CSS grid. It features a top filter panel and sorting options (alphabetical, review count, rating, favorites). Clicking a cultural site 'Card' centers the map on its marker and opens its details in the side panel (Figure 16).

The My Account Pages serve as a central user management hub, adapting to user roles ('user' or 'admin') (Figure 6, Figure 7).

- User Role: Features six sections: Profile Card (basic info, logout), Profile (account details modification, Figure 8), Favorite Sites (view/remove bookmarks, Figure 9), Reviews (check, edit, delete user reviews, Figure 9), My Proposals (track submitted proposals, Figure 11), and Delete Account (comprehensive hard deletion, removing account and associated data like reviews, favorites, and proposedBy fields via MongoDB sessions and transactions, Figure 17).
- Admin Role: Shares identical Profile Card, Profile editing, Favorite Sites, Reviews, and Delete Account functionalities with user roles (Figure 7).

Distinguished by View Proposals (instead of My Proposals) to view, accept, or reject all user proposals (Figure 12), and a Check Users section to manage all registered users and change their roles (Figure 13).

2.2 Database Structure (MongoDB collections)

This application leverages a **MongoDB** database named **dbw**, which is structured into five essential collections: culturalsites, excludesourceids, proposals, reviews, and users. The **culturalsites** collection houses information on Chemnitz cultural sites, primarily sourced from the OSM database via the Overpass API; while the data here differs slightly from the original API output, the raw original data is preserved within an originalTags field. The **excludesourceids** collection plays a crucial role in preventing the re-addition of deleted cultural sites, ensuring that once a site is removed—either by a user's proposal or an admin's direct action—it isn't re-registered through automated updates or by subsequent user/admin attempts. All user-generated proposals are recorded in the **proposals** collection, with each proposal schema detailing the target cultural site for addition, update, or deletion, along with proposer information, proposal type, content, status, and any administrative comments. The **reviews** collection stores all user-submitted reviews, linking each review to its respective cultural site via the objectId of the culturalsites document, and includes fields such as culturalSite, user, rating, comment, and createdAt. Finally, the **users** collection comprehensively stores all user information, encompassing fields like username, email, googleId, profileImage, role (e.g., user, admin), favoriteSites, active status, and a bio.

2.3 Backend Components (API & Logic)

Data Preprocessing

To improve the cultural sites dataset, I expanded the Overpass API query with tags like "tourism"="attraction", "amenity"="arts_centre", "amenity"="community_centre", "amenity"="library", and "amenity"="cinema". This refined query increased the dataset from 384 to approximately 470 locations. The culturalSiteSchema was then built based on an analysis of element tags using /backend/scripts/countProperties.js.

For data enrichment, osmDataProcessor.js in /backend/utils handles missing name, category, and address tags. determineCulturalSiteAddress performs reverse geocoding via the Nominatim API for missing addresses. mapCulturalSiteCategory assigns one of 10 distinct categories, primarily based on 'amenity' or 'tourism' tags, with custom logic for exceptions.

Data Synchronization (/backend/services)

The /backend/services directory manages the culturalsites collection and its synchronization. A function automatically adds a cultural site's identifier to excludesourceids upon removal from culturalsites.

More critically, overpassService.js (using node-cron in app.js) dispatches baseCulturalSiteQuery (from /backend/config/osmData.js) to the Overpass API weekly. New sourceId entries are added automatically to MongoDB, while originalTags field from existing ones are updated if element.tags differ from originalTags. Importantly, the service prevents re-adding or updating sites whose sourceId is in excludesourceids, honoring deliberate deletions from user proposals or admin actions. All these changes are logged in /backend/logs.

Models (/backend/models)

Mongoose Schemas for each MongoDB collection are defined in separate files within /backend/models, ensuring a clear and organized data model structure.

API Structure (/backend/routes & /backend/controllers)

The backend follows a modular architecture with clear separation of concerns. /backend/routes contains individual Express routers for features like auth, culturalSites, proposal, review, and user, directing requests to the appropriate controllers. /backend/controllers houses the logic for each feature, including a dedicated errorController for centralized error handling. More detailed information is in the API documentation located in the Appendix.

Key Utilities (/backend/utils)

The /backend/utils directory contains essential utility functions, including those for asynchronous exception and error handling. It also utilizes the Turf.js library for geospatial operations and includes functions for transforming raw OSM data to fit the CulturalSite schema, employing reverse geocoding using Nominatim API.

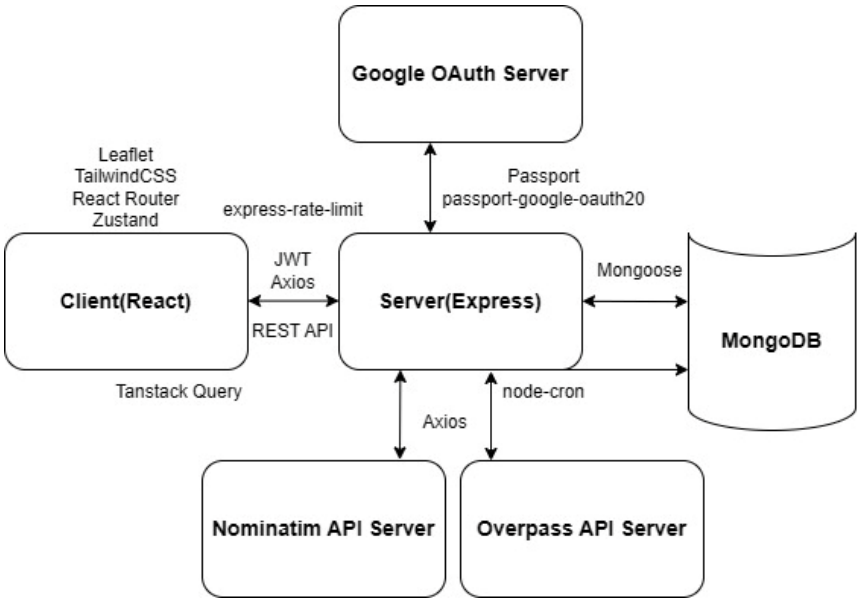

Figure 1: Overall Structure

3. Utilized Technologies

3.1 Frontend

For the frontend, React[1] was chosen to build a Single Page Application (SPA), aiming for better user experience. React Router[2] handles the client-side routing essential for SPA functionality, while Leaflet[3] and React-Leaflet[4] were integrated to create an interactive map, which is open source and lightweight. All communication with the backend is managed using Axios[5] which has automatic parsing, and Tanstack Query[6] is employed for caching, optimization, and centralized request management.

Regarding UI styling, Tailwind CSS[7] was selected for fast and efficient development. To dynamically generate a context menu when right-clicking on the map, @floating-ui/react[8] is used. react-icons[9] provides a wide array of icons, and Leaflet.markercluster[10] handles the styling for marker clusters on the map.

Additionally, other libraries were incorporated to enhance specific functionalities: lodash.debounce[11] ensures a smooth and natural debounce effect for the search feature, Zustand[12] manages global state within the browser with less boilerplate, and @tailwindcss/vite[13] facilitates the use of Tailwind CSS in the Vite environment.

3.2 Backend

The backend of the application is built using Express.js[14], consistent with the curriculum covered in the course. For seamless communication with external services like the Overpass API[26] and Nominatim API[27], Axios[5] was chosen for its clean error handling and efficient request management.

For robust JWT user authentication via Google OAuth 2.0, several libraries were integrated: passport-google-oauth20[15] and passport[16] handle the OAuth flow, while jsonwebtoken[17] is used for creating and verifying JSON Web Tokens, and cookie-parser[18] parse the cookie.

To securely store sensitive information, dotenv[19] is employed, allowing critical details to be kept in a .env file. cors[20] was used to address cross-origin resource sharing issues. Mongoose[21] provides an object data modeling (ODM) layer, making it significantly easier to interact with MongoDB data. For performing periodic automatic updates, the node-cron[22] library is utilized. The Express-Rate-Limit Library[23] was used to prevent excessive loads from a client side such as DDOS. Lastly, the Turf.js library[24] is for easier geospatial operations, enabling checks to determine if coordinates fall within the boundaries of Chemnitz or if two coordinates are within a specified distance of each other.

3.3 Database

MongoDB[25] was chosen as the database solution for several key reasons, particularly given the nature of OSM data obtained via the Overpass API[26]. OSM

data comes in various structures, including **nodes, ways, and relations**, with each element capable of possessing **dynamically changing tags**. MongoDB's **document-oriented model** is highly flexible, allowing it to directly store **GeoJSON formats compatible with OSM**, which is a significant advantage. Furthermore, using MongoDB offered the valuable opportunity to apply the **technologies learned in the course**, reinforcing practical implementation skills.

While MongoDB is a schemaless database, the Mongoose library[21] was used on the backend to define a clearer schema and leverage features like validation, providing a more structured approach to data management.

3.4 Technology References

[1] https://react.dev/

[2] https://reactrouter.com/

[3] https://leafletjs.com/

[4] https://react-leaflet.js.org/

[5] https://axios-http.com/

[6] https://tanstack.com/

[7] https://tailwindcss.com/

[8] https://floating-ui.com/

[9] https://react-icons.github.io/react-icons/

[10] https://github.com/Leaflet/Leaflet.markercluster

[11] https://www.npmjs.com/package/lodash.debounce

[12] https://zustand-demo.pmnd.rs/

[13] https://www.npmjs.com/package/@tailwindcss/vite

[14] https://expressjs.com/

[15] https://www.passportjs.org/packages/passport-google-oauth20/

[16] https://www.passportjs.org/

[17] https://github.com/auth0/node-jsonwebtoken

[18] https://github.com/expressjs/cookie-parser

[19] https://github.com/motdotla/dotenv

[20] https://github.com/expressjs/cors

[21] https://mongoosejs.com/

[22] https://github.com/node-cron/node-cron

[23] https://github.com/express-rate-limit/express-rate-limit

[24] https://turfjs.org/

[25] https://www.mongodb.com/

[26] https://wiki.openstreetmap.org/wiki/Overpass_API

[27] https://nominatim.org/release-docs/latest/api/Reverse/

5. Appendix

5.1 Figures

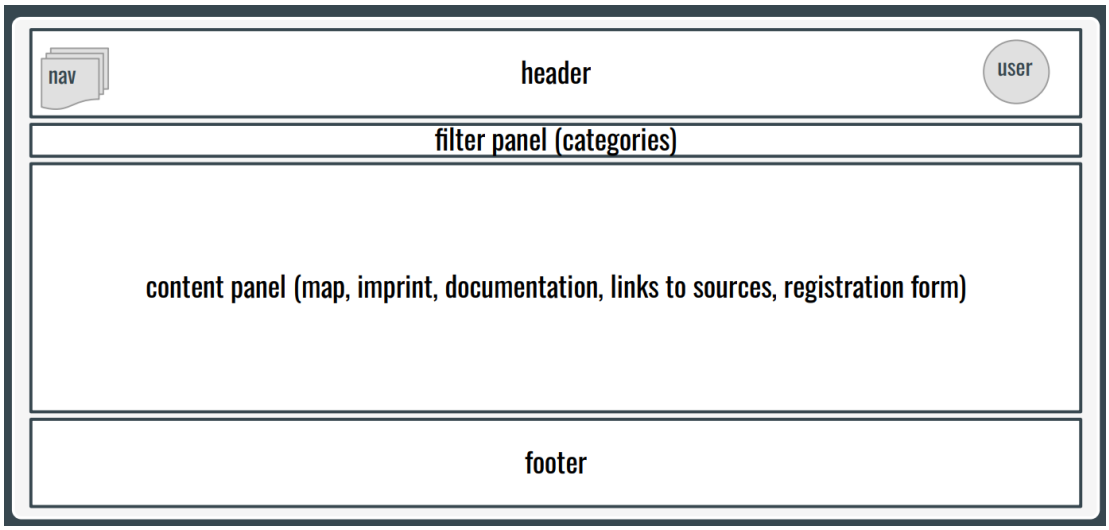

Figure 2: Project Task Front End UI Mockup

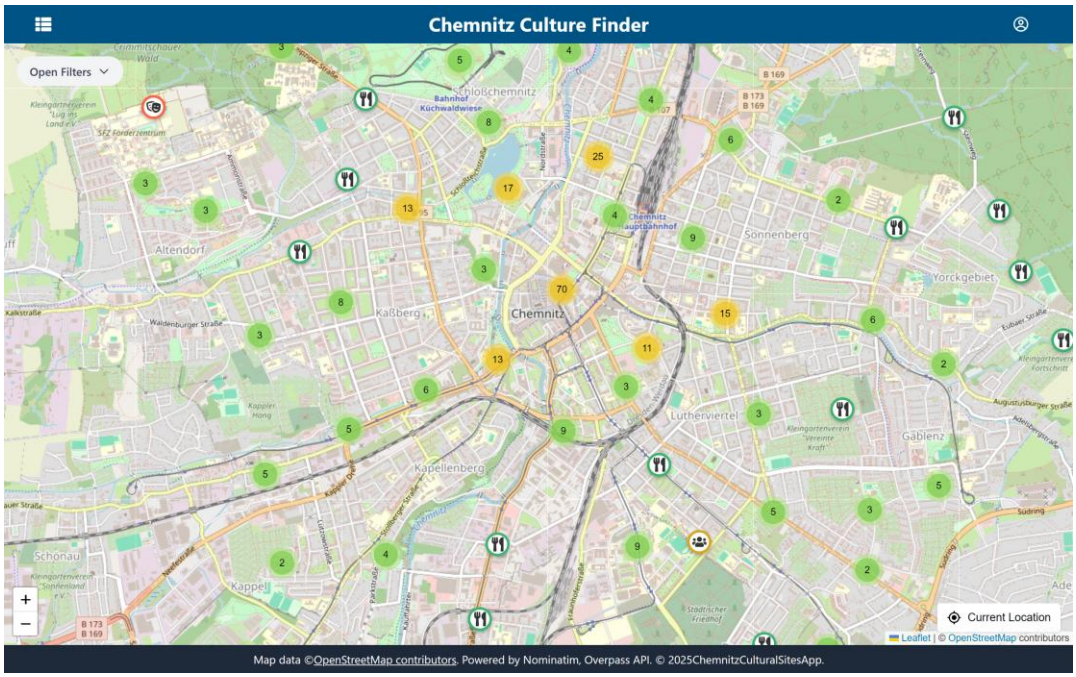

Figure 3: Home Page Layout

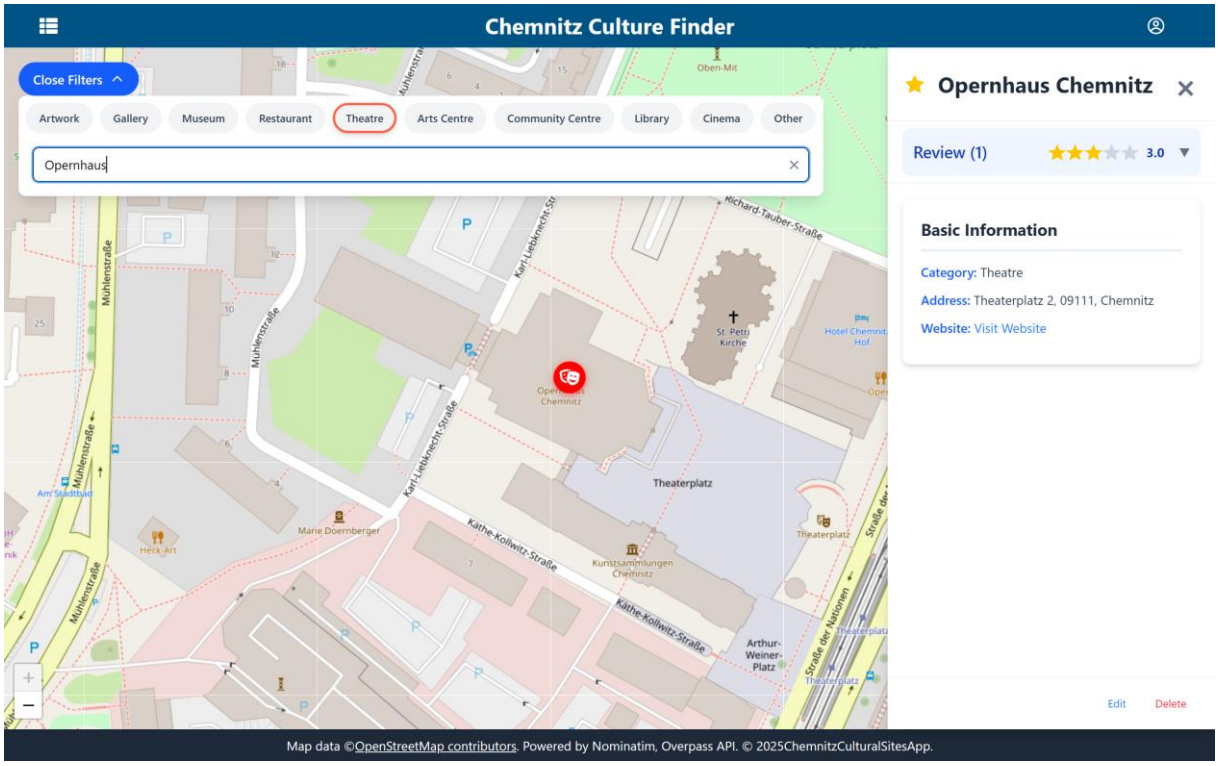

Figure 4: Filter Panel(categories, search) and Side Panel(detailed information)

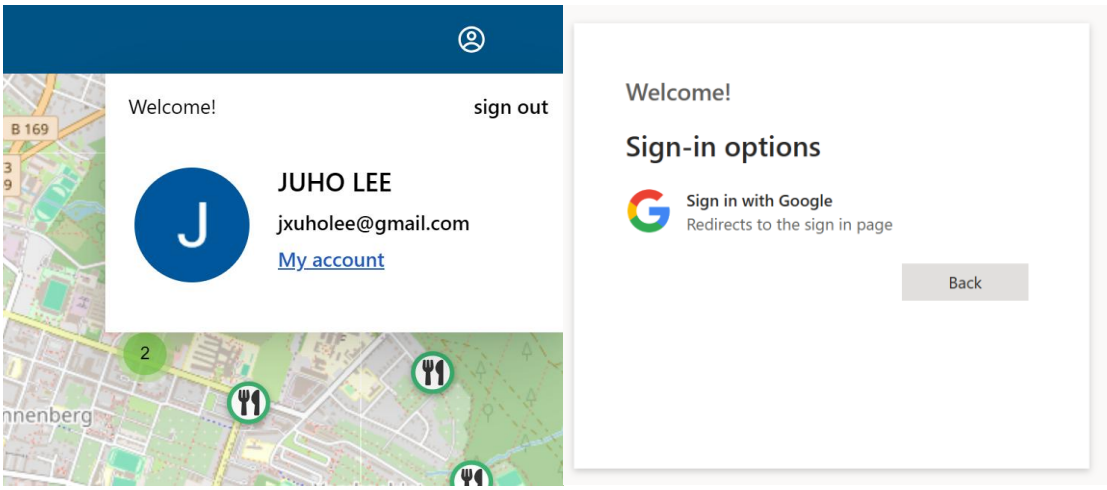

Figure 5: Account Manager Panel, Sign-In page

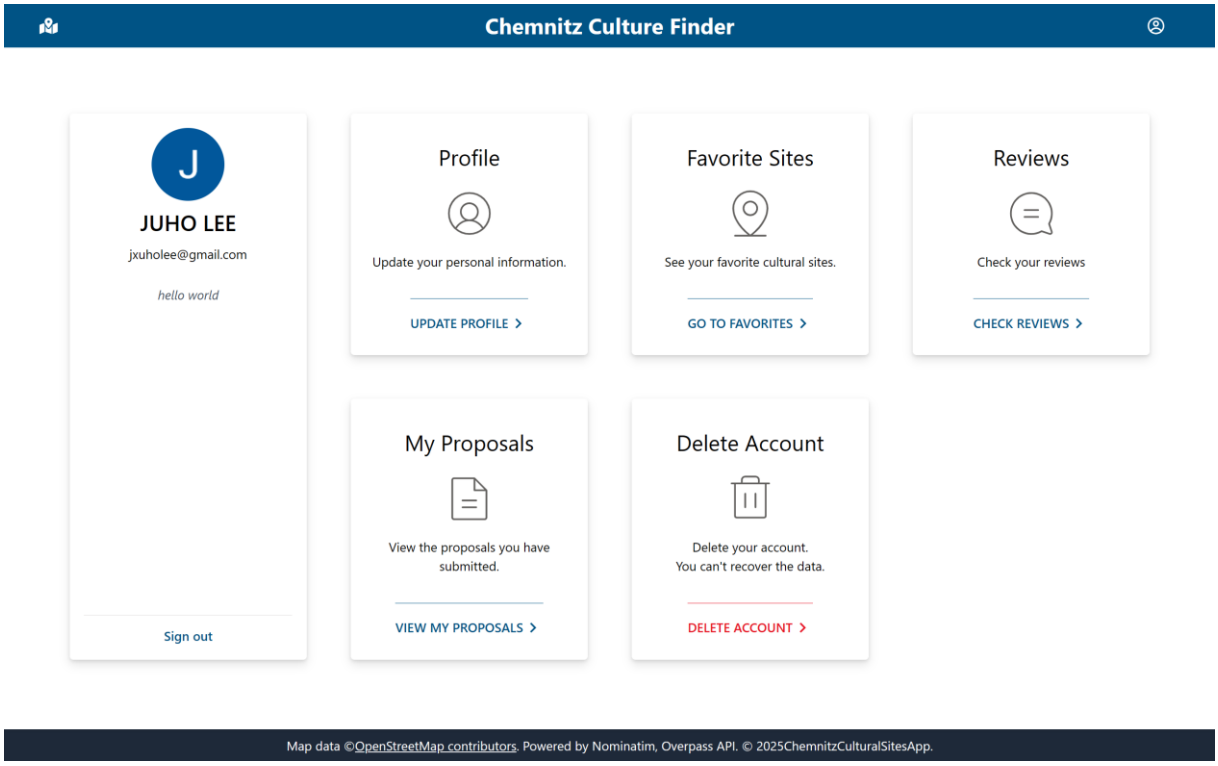

Figure 6: my-account page(user)

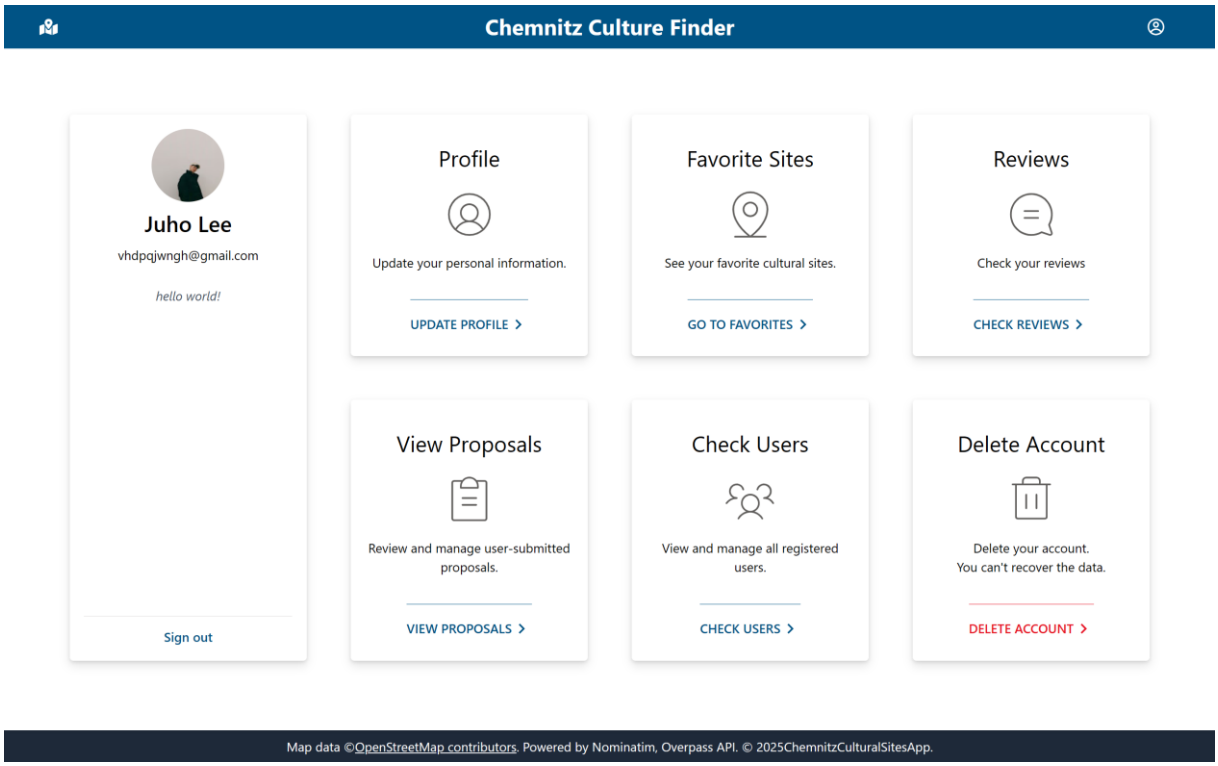

Figure 7: my-account page(admin)

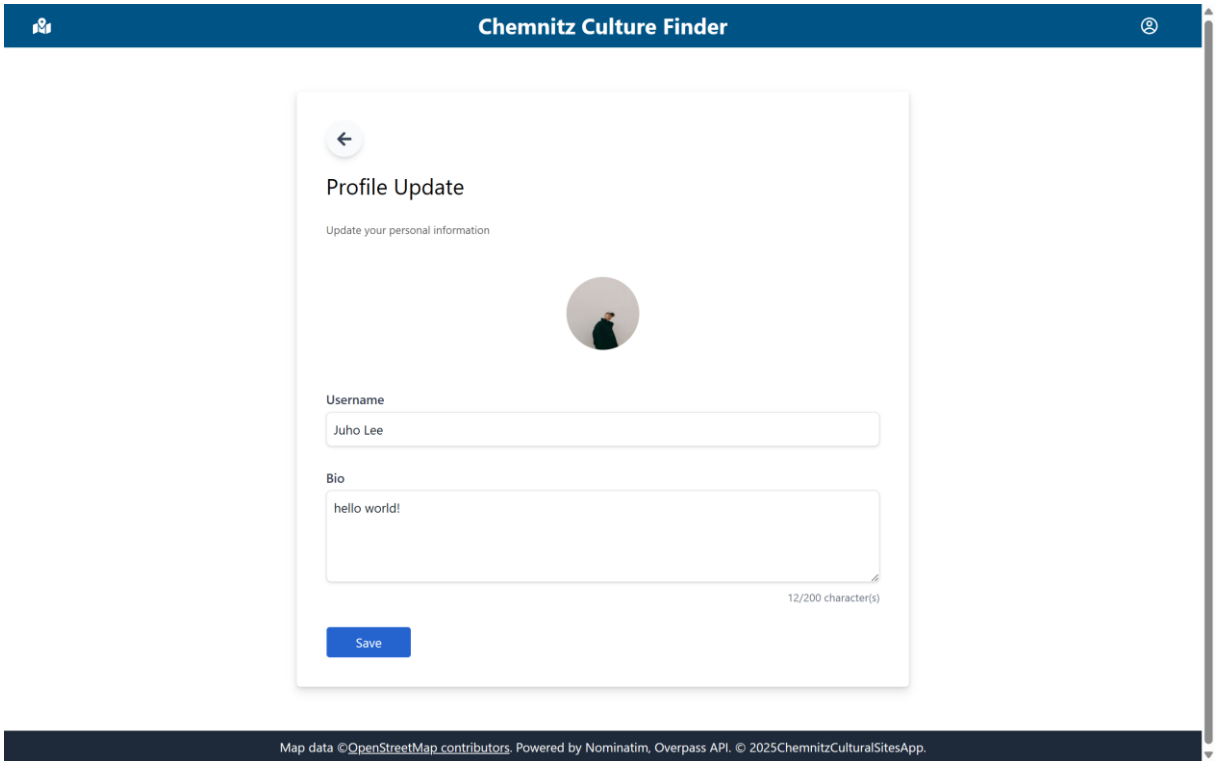

Figure 8: update-profile page

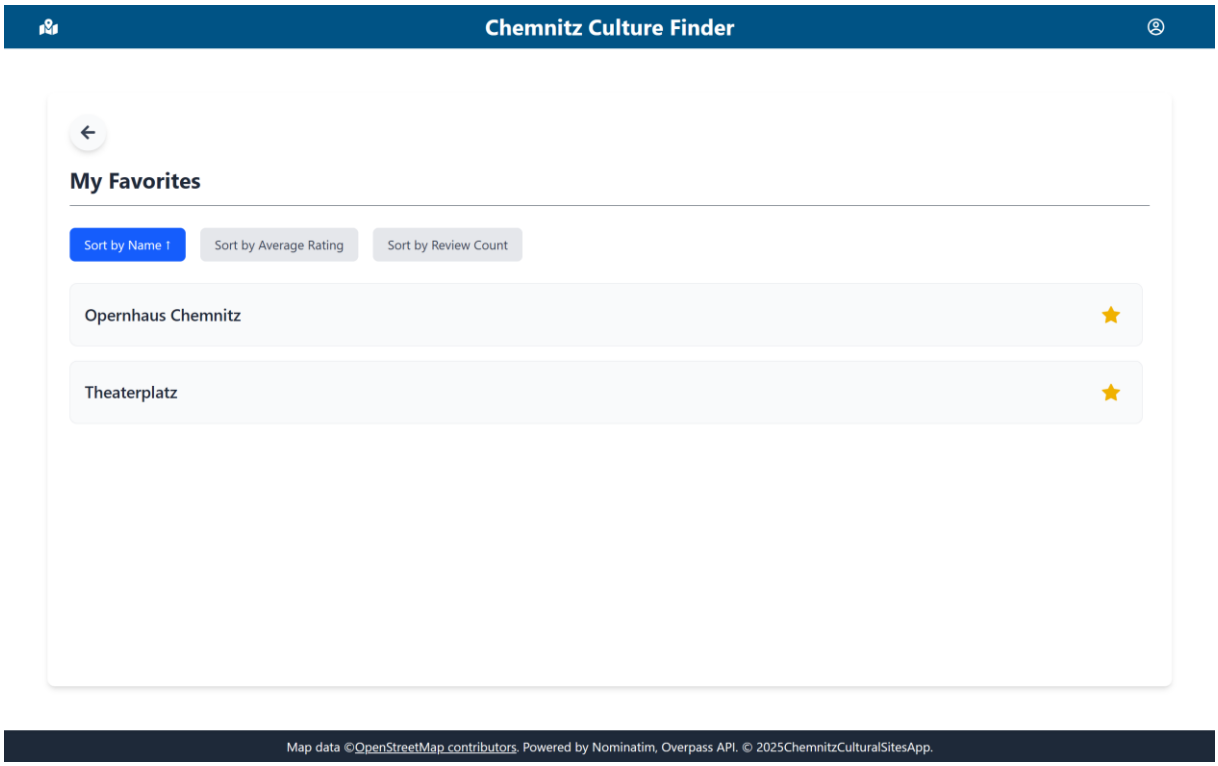

Figure 9: favorite-sites page

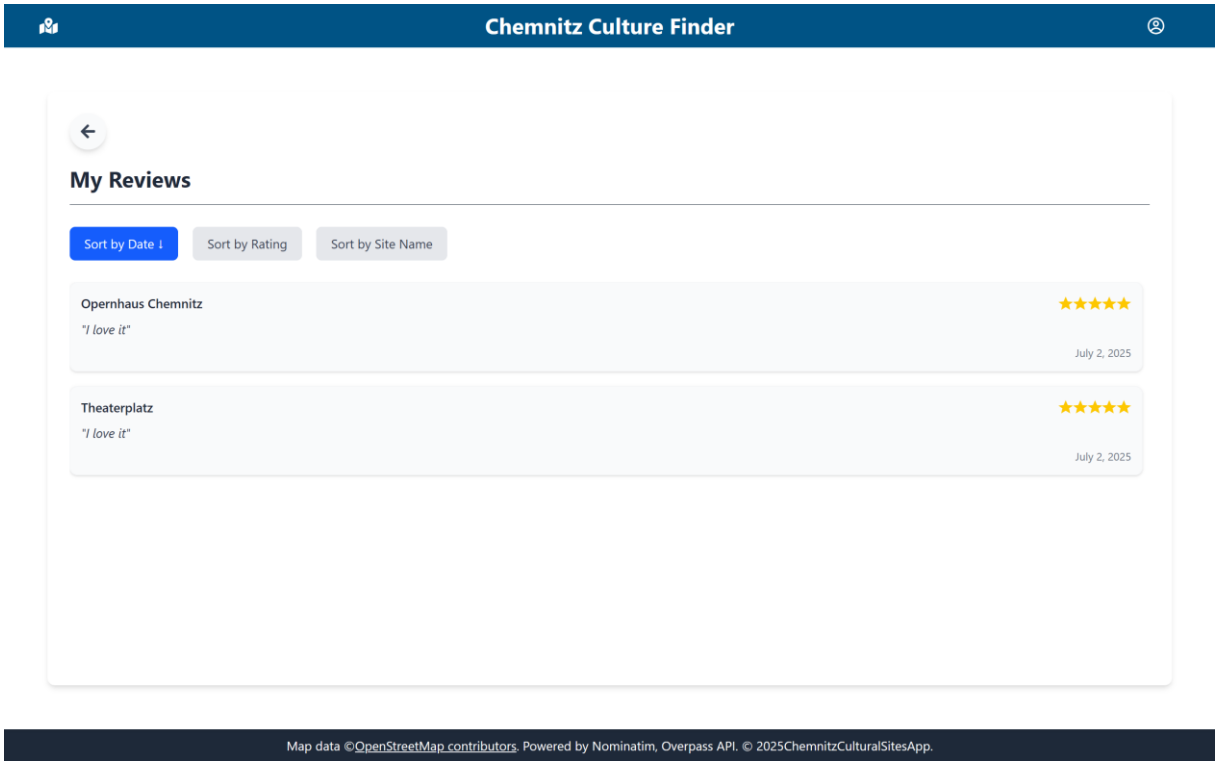

Figure 10: reviews page

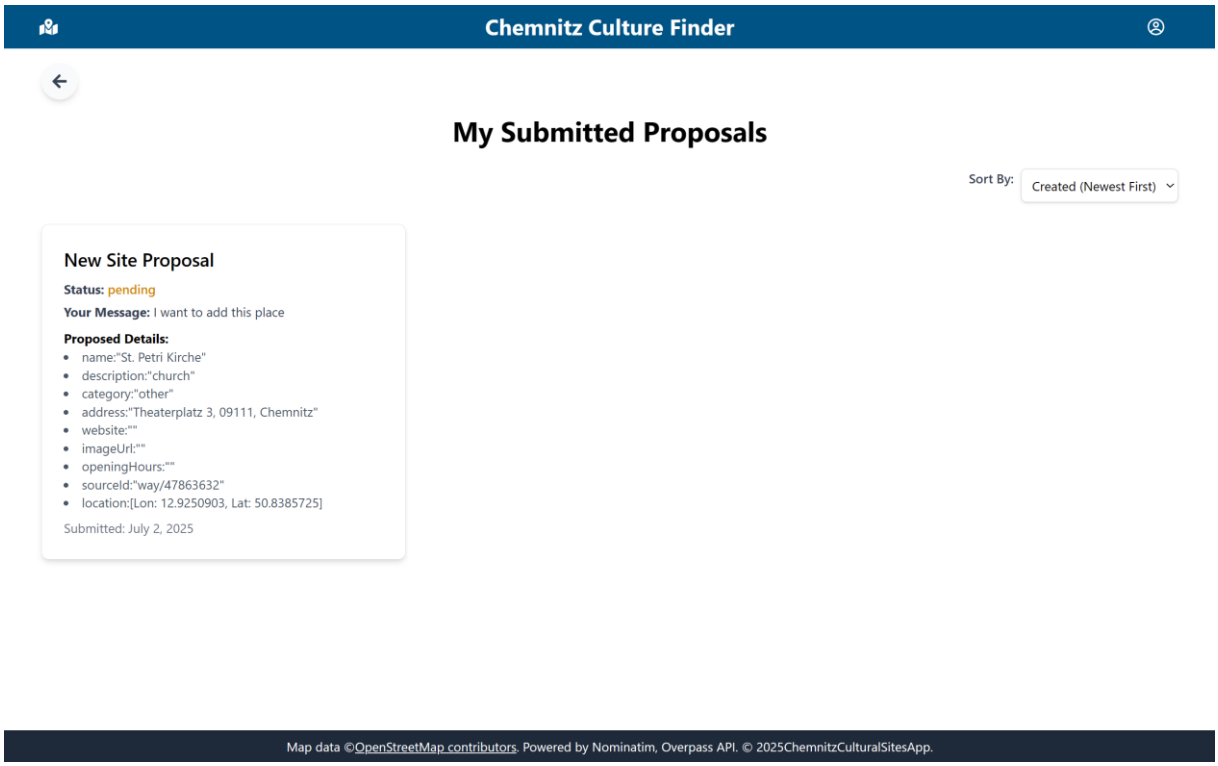

Figure 11: my proposals page(user)

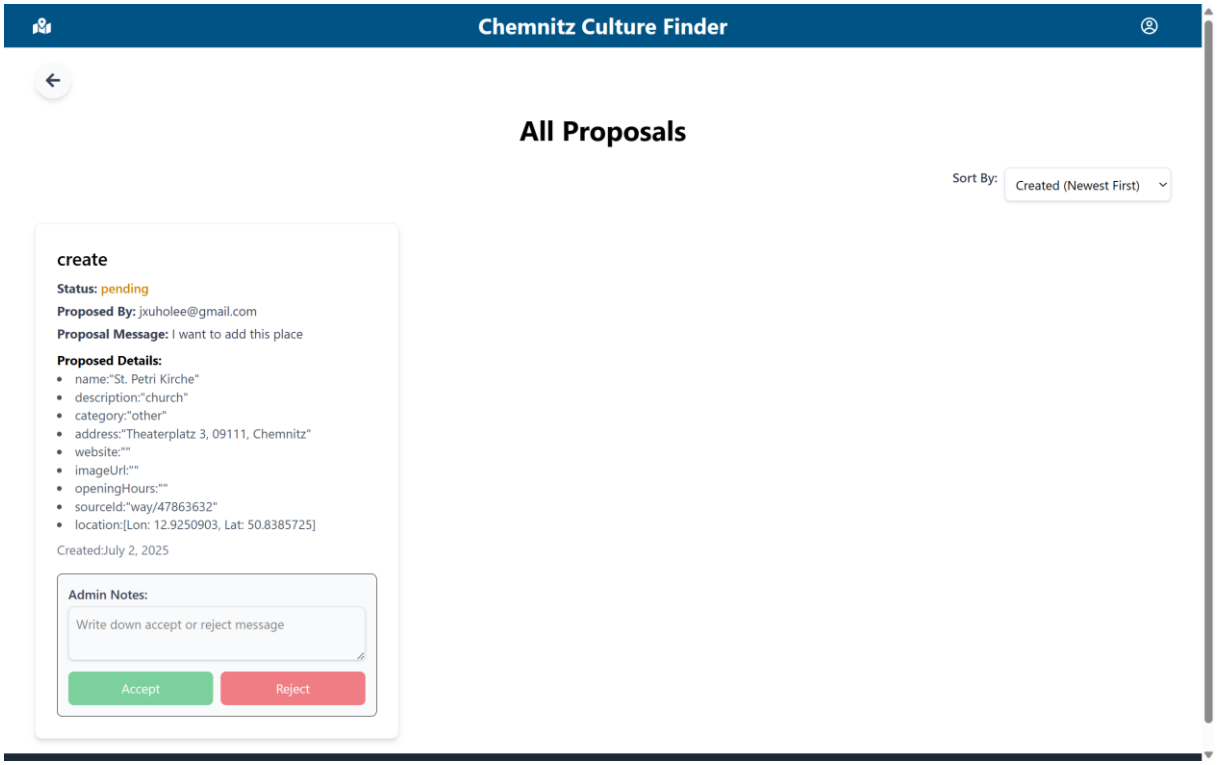

Figure 12: proposals page(admin)

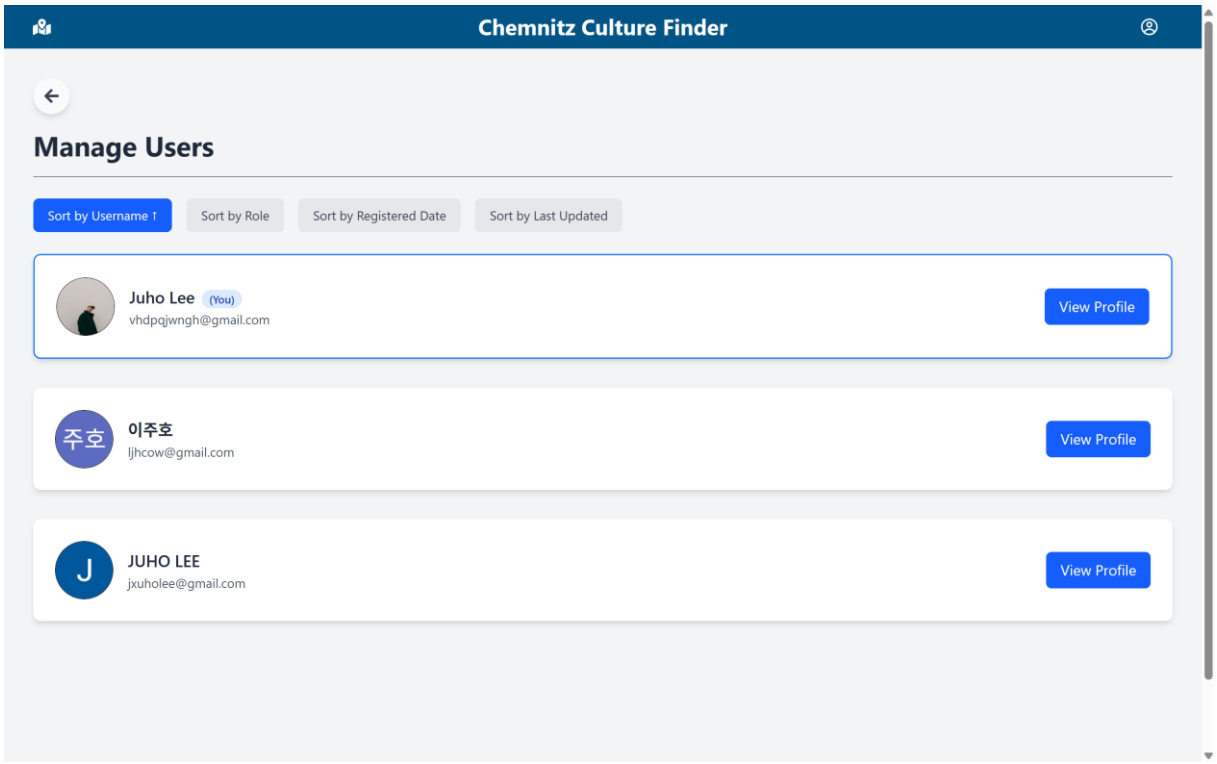

Figure 13: users page(admin)

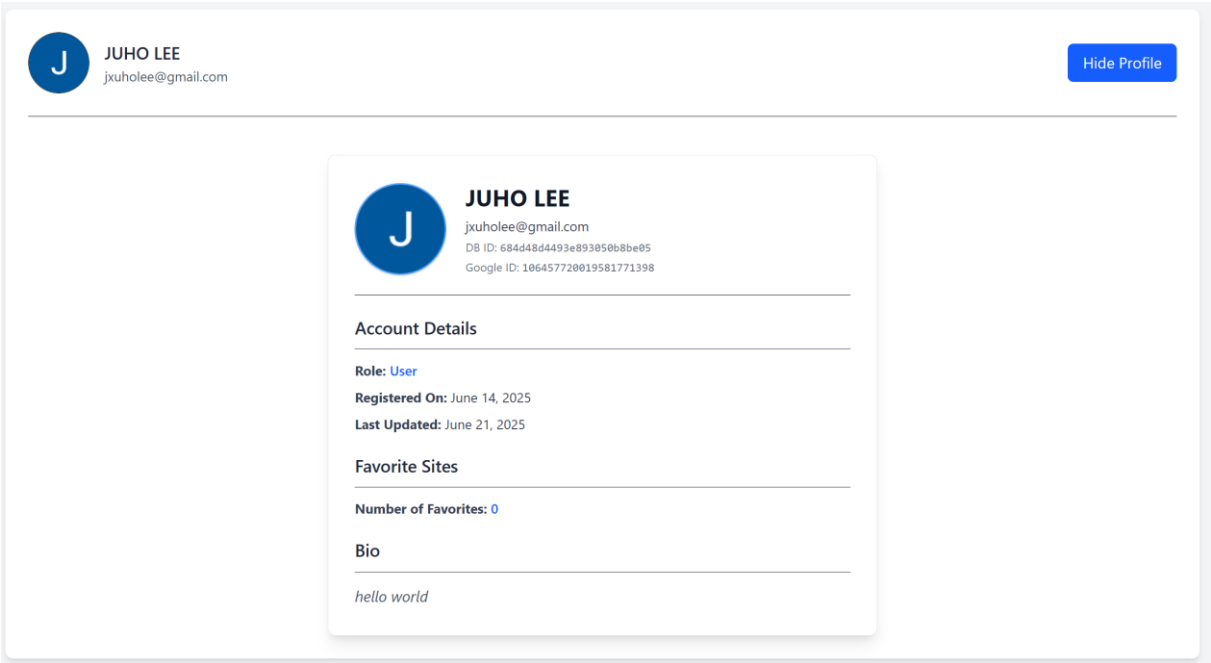

Figure 14: users page – view profile(admin)

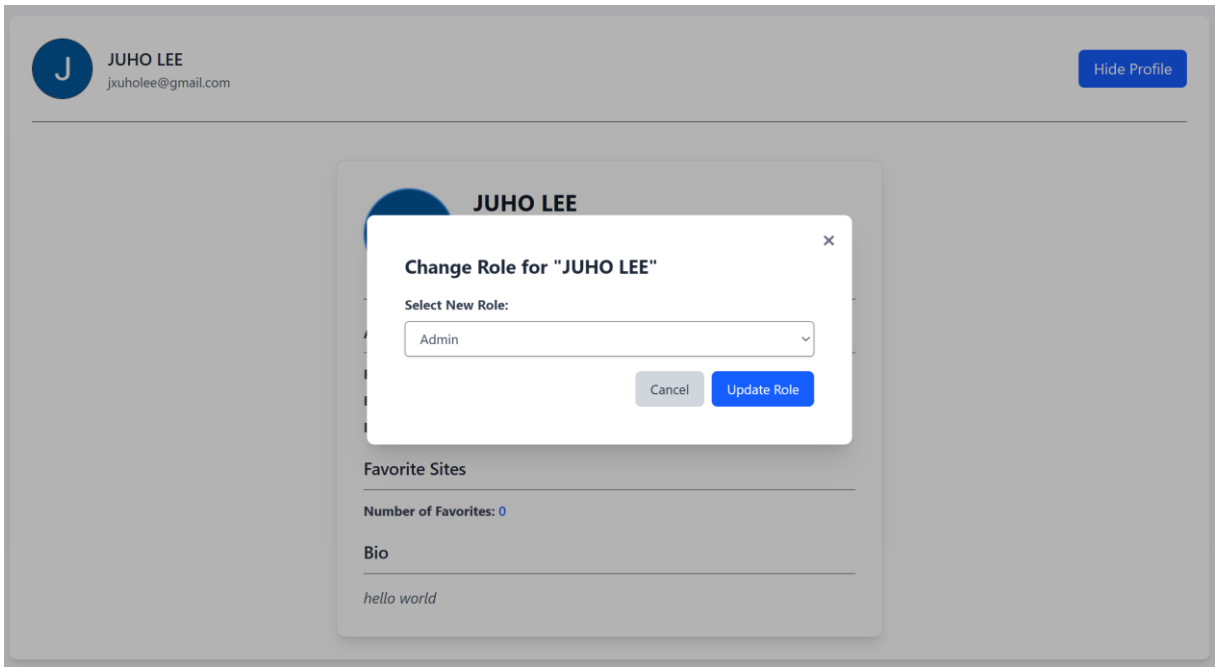

Figure 15: users page – change the role(admin)

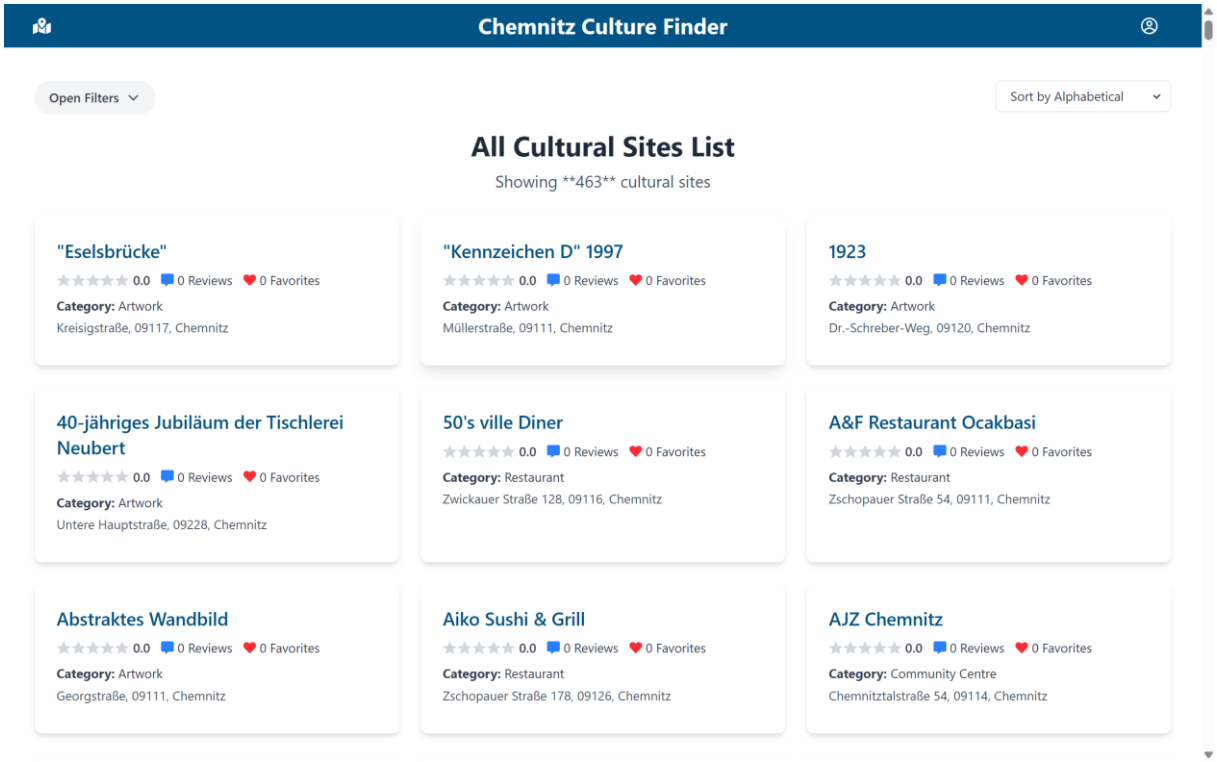

Figure 16: list page

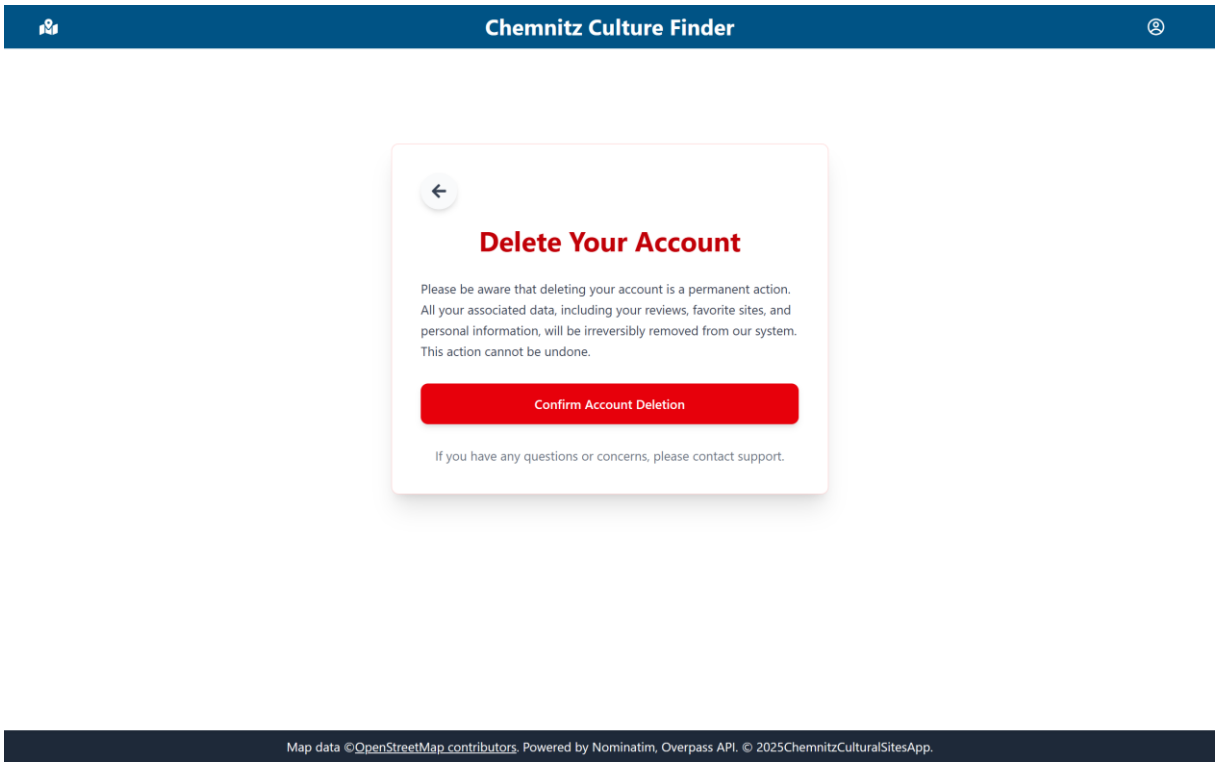

Figure 17: delete-account page

Figure 18: Suggest new place context menu on the map(user)

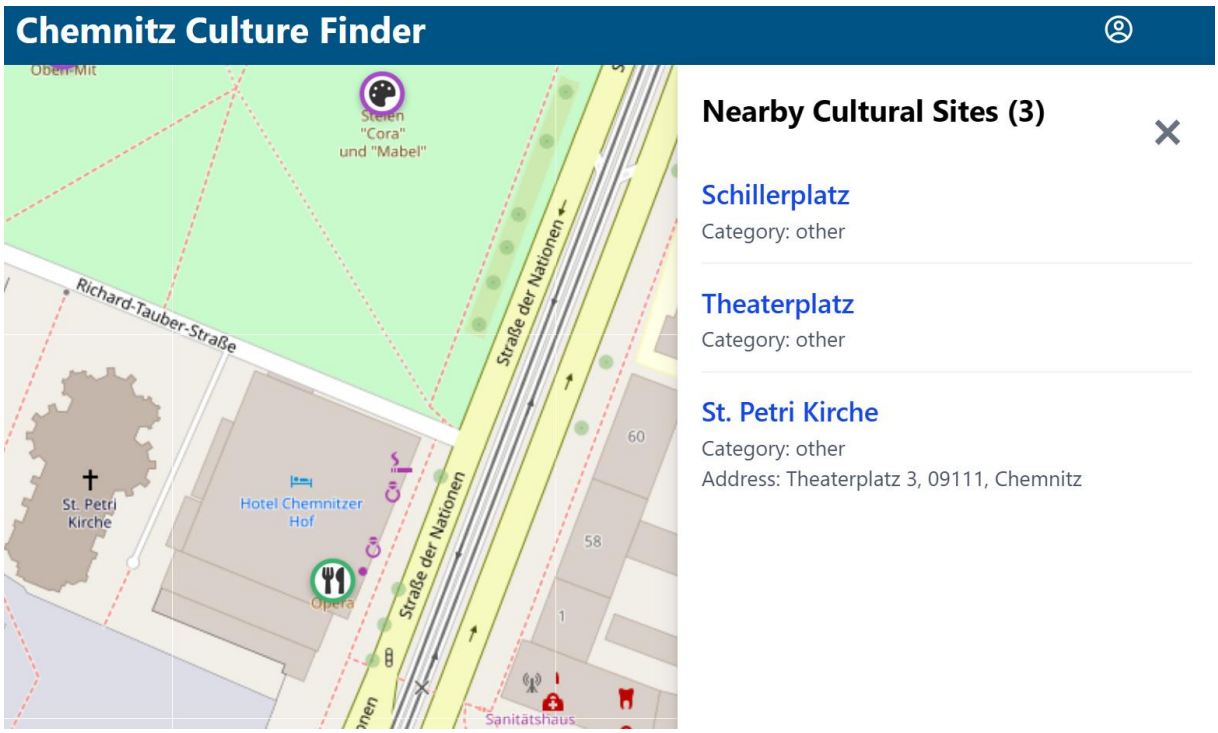

Figure 19: Nearby Cultural sites

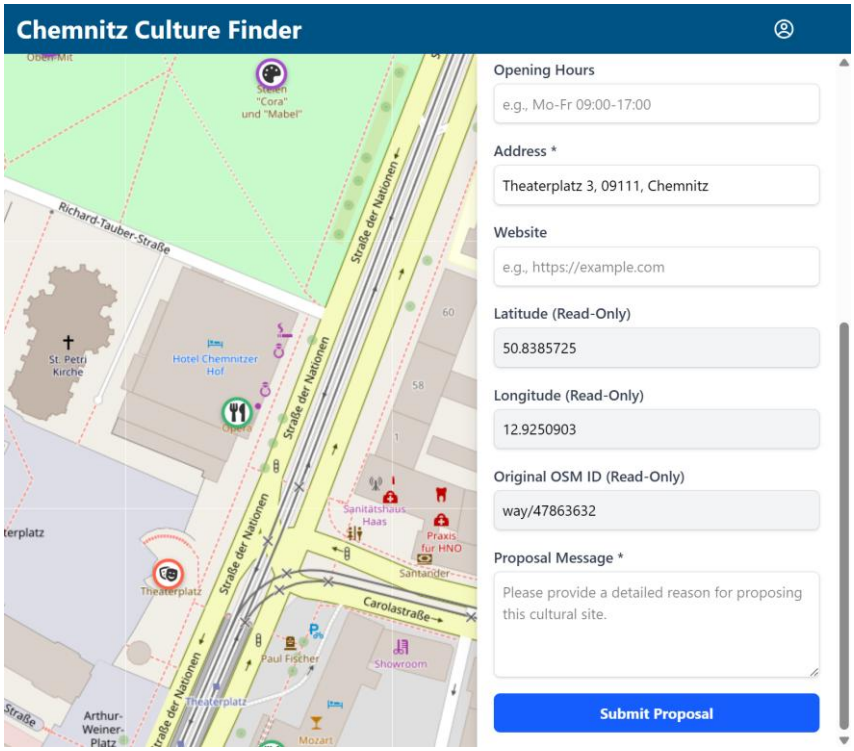

Figure 20: New cultural site proposal submission form

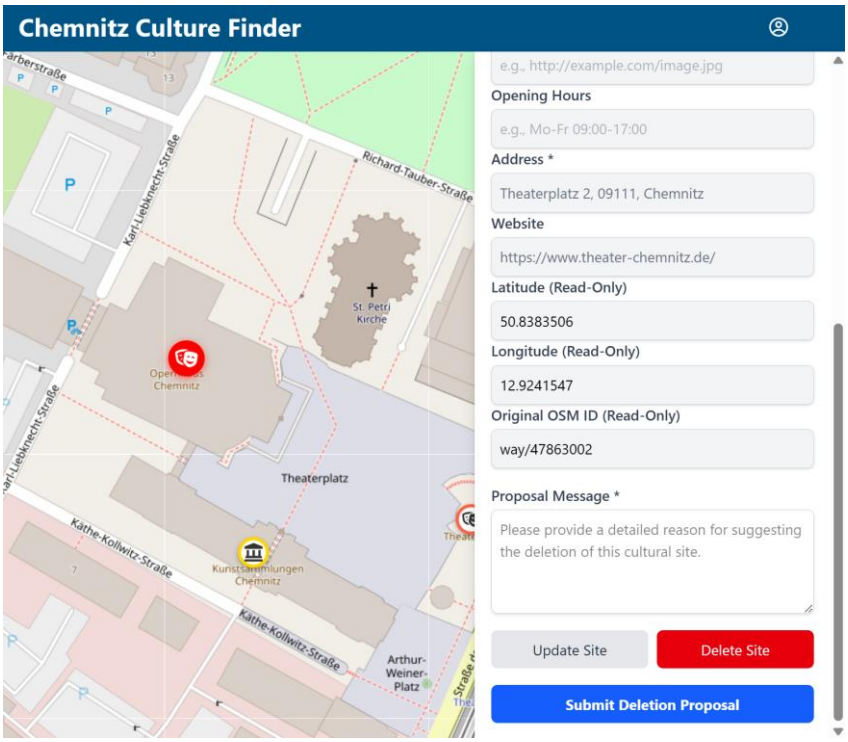

Figure 21: update/delete proposal submission form(for existing cultural site)

5.2 Web Service API Documentation

The openapi.yaml same as below can be accessed from `/backend/public` or from http://localhost:5000/api-docs/. (swagger-ui-express library)

```yaml
openapi: 3.1.0
info:
  title: Cultural Site API
  version: 1.0.0
  description: API for managing cultural sites, proposals, reviews,
and user data.
servers:
  - url: http://localhost:5000/api/v1
    description: Local Development Server
tags:
  - name: Authentication
    description: User authentication operations
  - name: Cultural Sites
    description: Operations related to cultural sites
  - name: Reviews
    description: Operations related to reviews of cultural sites
  - name: Proposals
    description: Operations related to proposals for cultural sites
  - name: Users
    description: Operations related to user management
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    CulturalSite:
      type: object
      properties:
```

```yaml
      _id:
        type: string
        description: Unique identifier for the cultural site.
      name:
        type: string
        description: Name of the cultural site.
        minLength: 2
        maxLength: 100
      description:
        type: string
        description: Description of the cultural site.
        maxLength: 1000
      category:
        type: string
        description: Category of the cultural site.
        enum:
          - artwork
          - gallery
          - museum
          - restaurant
          - theatre
          - arts_centre
          - community_centre
          - library
          - cinema
          - other
      location:
        type: object
        properties:
          type:
            type: string
            enum: ['Point']
            description: GeoJSON type.
          coordinates:
```

```yaml
              type: array
              items:
                type: number
              minItems: 2
              maxItems: 2
              description: |
                [longitude, latitude] coordinates. Longitude must be
between -180 and 180,
                latitude must be between -90 and 90.
          required:
            - type
            - coordinates
        address:
          type: string
          description: Address of the cultural site.
        website:
          type: string
          format: url
          description: Website URL of the cultural site.
        imageUrl:
          type: string
          format: url
          description: Image URL of the cultural site.
        openingHours:
          type: string
          description: Opening hours of the cultural site.
        licenseInfo:
          type: string
          default: "Data © OpenStreetMap contributors, ODbL."
          description: Licensing information for the data.
        sourceId:
          type: string
          description: OSM ID (node/way/relation ID) from Overpass
API. (node/1234567890)
```

```yaml
        reviews:
          type: array
          items:
            type: string
            description: ObjectId of a Review.
        favoritesCount:
          type: number
          format: int32
          minimum: 0
          default: 0
          description: Number of users who favorited this site.
        originalTags:
          type: object
          description: Original GeoJSON 'tags' object from the
source element from Overpass API response.
          additionalProperties: true
        proposedBy:
          type: string
          description: ObjectId of the User who proposed this site.
        registeredBy:
          type: string
          description: ObjectId of the User (admin) who registered
this site.
        active:
          type: boolean
          default: true
          description: Field for soft deletion (not currently
implemented).
      required:
        - name
        - category
        - location
        - address
        - sourceId
```

```yaml
      - originalTags
  Review:
    type: object
    properties:
      _id:
        type: string
        description: MongoDB ObjectId for the review.
      culturalSite:
        type: string
        description: ObjectId of the CulturalSite being reviewed.
      user:
        type: string
        description: ObjectId of the User who wrote the review.
      rating:
        type: number
        format: int32
        minimum: 1
        maximum: 5
        description: Rating given by the user (1 to 5).
      comment:
        type: string
        maxLength: 500
        description: Comment provided by the user.
      createdAt:
        type: string
        format: date-time
        description: Timestamp when the review was created.
      active:
        type: boolean
        default: true
        description: Field for soft deletion. (not implemented)
    required:
      - culturalSite
      - user
```

```
        - rating
    User:
      type: object
      properties:
        _id:
          type: string
          description: Unique identifier for the user.
        username:
          type: string
          maxLength: 20
          description: User's chosen username.
        email:
          type: string
          format: email
          description: User's email address.
        googleId:
          type: string
          description: Google ID if authenticated via Google.
        profileImage:
          type: string
          format: url
          description: URL to the user's profile image.
        role:
          type: string
          enum: ['user', 'admin']
          default: 'admin'
          description: User's role in the system. temporarily
default value is 'admin' to demonstrate the functions.
        active:
          type: boolean
          default: true
          description: Field for soft deletion.
        favoriteSites:
          type: array
```

```yaml
          items:
            type: string
            description: ObjectId of a CulturalSite.
          description: List of cultural sites marked as favorites by
the user.
        bio:
          type: string
          maxLength: 200
          description: User's short biography.
      required:
        - email
        - googleId
    Proposal:
      type: object
      properties:
        _id:
          type: string
          description: Unique identifier for the proposal.
        culturalSite:
          type: string
          description: ObjectId of the CulturalSite being proposed
for modification or deletion. Not required for 'create' proposals.
        proposedBy:
          type: string
          description: ObjectId of the User who submitted the
proposal.
        proposalType:
          type: string
          enum: ['create', 'update', 'delete']
          description: Type of proposal (create, update, or delete).
        proposedChanges:
          type: object
          description: Proposed changes to the cultural site.
          additionalProperties: true
```

```yaml
        proposalMessage:
          type: string
          maxLength: 500
          description: Optional message from the proposer.
        status:
          type: string
          enum: ['pending', 'accepted', 'rejected']
          default: 'pending'
          description: Current status of the proposal.
        reviewedBy:
          type: string
          description: ObjectId of the User (admin) who reviewed the
proposal. Required if status is 'accepted' or 'rejected'.
        adminComment:
          type: string
          description: Comment from the reviewing administrator.
Required if status is 'accepted' or 'rejected'.
        reviewedAt:
          type: string
          format: date-time
          description: Timestamp when the proposal was reviewed.
      required:
        - proposedBy
        - proposalType
        - proposedChanges
paths:
  /auth/google:
    get:
      summary: Start Google authentication
      tags:
        - Authentication
      operationId: googleAuth
      description: Redirects the user to Google's authentication
page.
```

```
      responses:
        302:
          description: Redirect to Google for authentication.
  /auth/google/callback:
    get:
      summary: Google authentication callback
      tags:
        - Authentication
      operationId: googleAuthCallback
      description: Handles the callback from Google after
authentication. Issues a JWT on success.
      responses:
        302:
          description: Redirects to frontend on success or failure.
        200:
          description: Authentication successful, JWT issued (though
typically a redirect happens).
        401:
          description: Google authentication failed.
  /auth/logout:
    post:
      summary: User logout
      tags:
        - Authentication
      operationId: logoutUser
      description: Deletes the JWT cookie to log out the user.
      responses:
        200:
          description: Successfully logged out.
  /cultural-sites/nearby-osm:
    get:
      summary: Get nearby cultural sites from OSM database for a
lon/lat coordinates
      tags:
```

```yaml
      - Cultural Sites
    operationId: getNearbyOsmCulturalSites
    security:
      - bearerAuth: []
    parameters:
      - in: query
        name: latitude
        schema:
          type: number
          format: float
        required: true
        description: Latitude for the center of the search.
      - in: query
        name: longitude
        schema:
          type: number
          format: float
        required: true
        description: Longitude for the center of the search.
      - in: query
        name: radius
        schema:
          type: number
          format: int32
          default: 1000 # Corrected: default is inside schema
        description: Search radius in meters.
    responses:
      200:
        description: A list of nearby cultural sites from OSM.
        content:
          application/json:
            schema:
              type: array
              items:
```

```yaml
                  $ref: '#/components/schemas/CulturalSite'
        400:
          description: Invalid latitude/longitude or location not
within Chemnitz.
        401:
          description: Unauthorized if no token is provided.
        403:
          description: Forbidden if user is not 'admin' or 'user'.
        500:
          description: Internal server error.
  /cultural-sites:
    get:
      summary: Get all cultural sites
      tags:
        - Cultural Sites
      operationId: getAllCulturalSites
      parameters:
        - in: query
          name: category
          schema:
            type: string
          description: Filter by cultural site category.
        - in: query
          name: name
          schema:
            type: string
          description: Filter by cultural site name (partial match).
      responses:
        200:
          description: A list of cultural sites.
          content:
            application/json:
              schema:
                type: array
```

```yaml
                items:
                  $ref: '#/components/schemas/CulturalSite'
      500:
        description: Internal server error.
  post:
    summary: Save a cultural site to the database
    tags:
      - Cultural Sites
    operationId: saveCulturalSiteToDb
    security:
      - bearerAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CulturalSite'
          examples:
            newCulturalSite:
              value:
                name: "Example Museum"
                description: "A wonderful museum of modern art."
                category: "Museum"
                location:
                  type: "Point"
                  coordinates: [12.345, 54.321]
                address: "123 Museum St, City"
                website: "http://examplemuseum.com"
                sourceId: "osm_node_12345"
                originalTags:
                  amenity: "museum"
                  name: "Example Museum"
    responses:
      201:
```

```
          description: Cultural site successfully saved.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/CulturalSite'
        400:
          description: Invalid input (e.g., missing fields, invalid
location, not in Chemnitz).
        401:
          description: Unauthorized.
        403:
          description: Forbidden (only admins can add sites, or
sourceId is in exclusion list).
        409:
          description: Conflict (e.g., sourceId already registered).
        500:
          description: Internal server error during saving.
  /cultural-sites/{id}:
    get:
      summary: Get a cultural site by ID
      tags:
        - Cultural Sites
      operationId: getCulturalSiteById
      parameters:
        - in: path
          name: id
          schema:
            type: string
          required: true
          description: Unique ID of the cultural site.
      responses:
        200:
          description: Details of the cultural site.
          content:
```

```yaml
          application/json:
            schema:
              $ref: '#/components/schemas/CulturalSite'
      400:
        description: Invalid cultural site ID format.
      404:
        description: Cultural site not found.
      500:
        description: Internal server error.
  put:
    summary: Update a cultural site by ID (Admin only)
    tags:
      - Cultural Sites
    operationId: updateCulturalSiteById
    security:
      - bearerAuth: []
    parameters:
      - in: path
        name: id
        schema:
          type: string
        required: true
        description: Unique ID of the cultural site.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/CulturalSite'
          examples:
            updateCulturalSite:
              value:
                description: "Updated description of the museum."
                openingHours: "9 AM - 5 PM"
```

```yaml
    responses:
      200:
        description: Cultural site updated successfully.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CulturalSite'
      400:
        description: Invalid input.
      401:
        description: Unauthorized.
      403:
        description: Forbidden, only admins can update sites.
      404:
        description: Cultural site not found.
  delete:
    summary: Delete a cultural site by ID (Admin only)
    tags:
      - Cultural Sites
    operationId: deleteCulturalSiteById
    security:
      - bearerAuth: []
    parameters:
      - in: path
        name: id
        schema:
          type: string
        required: true
        description: Unique ID of the cultural site.
    responses:
      204:
        description: Cultural site successfully deleted.
      401:
        description: Unauthorized.
```

```
        403:
          description: Forbidden (only admins can delete sites).
        404:
          description: Cultural site not found.
        409:
          description: Conflict (e.g., sourceId already exists in
exclusion list).
        500:
          description: Internal server error during deletion.
  /cultural-sites/{culturalSiteId}/reviews:
    get:
      summary: Get all reviews for a specific cultural site
      tags:
        - Reviews
      operationId: getAllReviewsForCulturalSite
      parameters:
        - in: path
          name: culturalSiteId
          schema:
            type: string
          required: true
          description: Unique ID of the cultural site.
      responses:
        200:
          description: A list of reviews for the cultural site.
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Review'
        400:
          description: Invalid input parameter.
        404:
```

```yaml
        description: The specified cultural site does not exist.
      500:
        description: Internal server error.
  post:
    summary: Create a review for a specific cultural site
    tags:
      - Reviews
    operationId: createReviewForCulturalSite
    security:
      - bearerAuth: []
    parameters:
      - in: path
        name: culturalSiteId
        schema:
          type: string
        required: true
        description: Unique ID of the cultural site.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              rating:
                type: number
                format: int32
                minimum: 1
                maximum: 5
              comment:
                type: string
                maxLength: 500
            required:
              - rating
```

```
            examples:
              newReview:
                value:
                  rating: 4
                  comment: "A great place to visit!"
      responses:
        201:
          description: Review successfully created.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Review'
        400:
          description: Invalid input or user already reviewed this
site.
        401:
          description: Unauthorized.
        403:
          description: Forbidden, only 'user' or 'admin' can create
reviews.
        404:
          description: The cultural site for which the review is
being created does not exist.
        409:
          description: The user has already submitted a review for
this specific cultural site.
        500:
          description: internal server error
  /cultural-sites/{culturalSiteId}/reviews/{reviewId}:
    get:
      summary: Get a specific review for a cultural site
      tags:
        - Reviews
      operationId: getReviewByIdForCulturalSite
```

```yaml
    parameters:
      - in: path
        name: culturalSiteId
        schema:
          type: string
        required: true
        description: Unique ID of the cultural site.
      - in: path
        name: reviewId
        schema:
          type: string
        required: true
        description: Unique ID of the review.
    responses:
      200:
        description: Details of the review.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Review'
      404:
        description: Cultural site or review not found.
      500:
        description: Internal server error.
  patch:
    summary: Update a specific review
    tags:
      - Reviews
    operationId: updateReviewById
    security:
      - bearerAuth: []
    parameters:
      - in: path
        name: culturalSiteId
```

```yaml
        schema:
          type: string
        required: true
        description: Unique ID of the cultural site.
      - in: path
        name: reviewId
        schema:
          type: string
        required: true
        description: Unique ID of the review to update.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              rating:
                type: number
                format: int32
                minimum: 1
                maximum: 5
              comment:
                type: string
                maxLength: 500
          examples:
            updateReview:
              value:
                comment: "An updated comment."
    responses:
      200:
        description: Review updated successfully.
        content:
          application/json:
```

```
              schema:
                $ref: '#/components/schemas/Review'
        401:
          description: Unauthorized.
        403:
          description: Forbidden, only the review's owner can update
it.
        404:
          description: Review not found.
        500:
          description: Internal server error.
    delete:
      summary: Delete a specific review
      tags:
        - Reviews
      operationId: deleteReviewById
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: culturalSiteId
          schema:
            type: string
          required: true
          description: Unique ID of the cultural site.
        - in: path
          name: reviewId
          schema:
            type: string
          required: true
          description: Unique ID of the review to delete.
      responses:
        204:
          description: Review successfully deleted.
```

```yaml
        401:
          description: Unauthorized.
        403:
          description: Forbidden, only the review's owner or an
admin can delete it.
        404:
          description: Review not found.
        500:
          description: Internal server error.
  /proposals/my-proposals:
    get:
      summary: Get proposals submitted by the current user
      tags:
        - Proposals
      operationId: getProposalsByUserId
      security:
        - bearerAuth: []
      responses:
        200:
          description: A list of proposals submitted by the user.
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Proposal'
        401:
          description: Unauthorized.
        500:
          description: Internal server error.
  /proposals:
    post:
      summary: Create a new proposal (create, update, or delete)
      tags:
```

```
        - Proposals
      operationId: createProposal
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                culturalSite:
                  type: string
                  description: Cultural Site ID if proposalType is
'update' or 'delete'.
                proposalType:
                  type: string
                  enum: ['create', 'update', 'delete']
                proposedChanges:
                  type: object
                  description: |
                    For 'create' and 'update' types, details of the
changes.
                    For 'create', this should be the new cultural
site data directly.
                    For 'update', it should contain key-value pairs
of fields to update.
                    For 'delete', this should be empty.
                proposalMessage:
                  type: string
                  maxLength: 500
              required:
                - proposedBy # This field is injected by the server
                - proposalType
```

```
                - proposedChanges
            examples:
              createProposal:
                value:
                  proposalType: "create"
                  proposedChanges:
                    name: "New Museum Idea"
                    category: "Museum"
                    location:
                      type: "Point"
                      coordinates: [10.123, 20.456]
                    address: "New Street 1"
                    sourceId: "new_osm_id_xyz"
                    originalTags:
                      building: "yes"
                      name: "New Museum Idea"
              updateProposal:
                value:
                  culturalSite: "60c72b2f9b1d6c001c8e4d3a"
                  proposalType: "update"
                  proposedChanges:
                    description: "Updated description for the
museum."
                    openingHours: "10 AM - 6 PM"
              deleteProposal:
                value:
                  culturalSite: "60c72b2f9b1d6c001c8e4d3b"
                  proposalType: "delete"
                  proposedChanges: {}
      responses:
        201:
          description: Proposal successfully created.
          content:
            application/json:
```

```
              schema:
                $ref: '#/components/schemas/Proposal'
        400:
          description: Invalid input.
        401:
          description: Unauthorized.
        404:
          description: The target resource for the proposal was not
found.
        409:
          description: A conflict occurred, typically because a
similar proposal or resource already exists.
        500:
          description: Internal server error
    get:
      summary: Get all proposals (Admin only)
      tags:
        - Proposals
      operationId: getAllProposals
      security:
        - bearerAuth: []
      parameters:
        - in: query
          name: status
          schema:
            type: string
            enum: ['pending', 'accepted', 'rejected']
          description: Filter proposals by status.
        - in: query
          name: proposalType
          schema:
            type: string
            enum: ['create', 'update', 'delete']
          description: Filter proposals by type.
```

```yaml
      responses:
        200:
          description: A list of all proposals.
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Proposal'
        401:
          description: Unauthorized.
        403:
          description: Forbidden, only admins can view all
proposals.
        500:
          description: Internal server error.
  /proposals/{id}:
    get:
      summary: Get a specific proposal by ID (Admin only)
      tags:
        - Proposals
      operationId: getProposalById
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: id
          schema:
            type: string
          required: true
          description: Unique ID of the proposal.
      responses:
        200:
          description: Details of the proposal.
```

```yaml
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Proposal'
        401:
          description: Unauthorized.
        403:
          description: Forbidden, only admins can view proposals by
ID.
        404:
          description: Proposal not found.
        500:
          description: Internal server error.
  /proposals/{id}/accept:
    patch:
      summary: Accept a proposal (Admin only)
      tags:
        - Proposals
      operationId: acceptProposal
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: id
          schema:
            type: string
          required: true
          description: Unique ID of the proposal to accept.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
```

```yaml
            properties:
              adminComment:
                type: string
                description: Comment from the administrator.
            required:
              - adminComment
    responses:
      200:
        description: Proposal successfully accepted.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Proposal'
      400:
        description: Invalid input or proposal already reviewed.
      401:
        description: Unauthorized.
      403:
        description: Forbidden, only admins can accept proposals.
      404:
        description: Proposal not found.
      500:
        description: Internal server error.
  /proposals/{id}/reject:
    patch:
      summary: Reject a proposal (Admin only)
      tags:
        - Proposals
      operationId: rejectProposal
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: id
```

```yaml
        schema:
          type: string
        required: true
        description: Unique ID of the proposal to reject.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              adminComment:
                type: string
                description: Comment from the administrator.
            required:
              - adminComment
    responses:
      200:
        description: Proposal successfully rejected.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Proposal'
      400:
        description: Invalid input or proposal already reviewed.
      401:
        description: Unauthorized.
      403:
        description: Forbidden, only admins can reject proposals.
      404:
        description: Proposal not found.
      500:
        description: Internal server error.
  /users/updateRole/{userId}:
```

```yaml
    patch:
      summary: Update a user's role (Admin only)
      tags:
        - Users
      operationId: updateUserRole
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: userId
          schema:
            type: string
          required: true
          description: Unique ID of the user whose role is to be
updated.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                role:
                  type: string
                  enum: ['user', 'admin']
              required:
                - role
            examples:
              changeToAdmin:
                value:
                  role: "admin"
      responses:
        200:
          description: User role updated successfully.
```

```yaml
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'
      400:
        description: Invalid input.
      401:
        description: Unauthorized.
      403:
        description: Forbidden, only admins can update user roles.
      404:
        description: User not found.
      500:
        description: Internal server error.
  /users/me:
    get:
      summary: Get current user's profile
      tags:
        - Users
      operationId: getMe
      security:
        - bearerAuth: []
      responses:
        200:
          description: Current user's profile details.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        401:
          description: Unauthorized.
        500:
          description: Internal server error.
    patch:
```

```yaml
      summary: Update current user's profile
      tags:
        - Users
      operationId: updateMe
      security:
        - bearerAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                username:
                  type: string
                  maxLength: 20
                profileImage:
                  type: string
                bio:
                  type: string
                  maxLength: 200
            examples:
              updateProfile:
                value:
                  username: "NewUsername"
                  bio: "My updated bio."
      responses:
        200:
          description: User profile updated successfully.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        400:
```

```yaml
          description: Invalid input.
        401:
          description: Unauthorized.
        500:
          description: Internal server error.
    delete:
      summary: Delete current user's account
      tags:
        - Users
      operationId: deleteMe
      security:
        - bearerAuth: []
      responses:
        200:
          description: User account successfully deleted.
        401:
          description: Unauthorized.
        404:
          description: can't find the user.
        500:
          description: Internal server error.
  /users/{userId}:
    get:
      summary: Get user information by ID
      tags:
        - Users
      operationId: getUserById
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: userId
          schema:
            type: string
```

```yaml
        required: true
        description: Unique ID of the user.
    responses:
      200:
        description: User profile details.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'
      401:
        description: Unauthorized.
      404:
        description: User not found.
      500:
        description: Internal server error.
/users:
  get:
    summary: Get all user information (Admin only)
    tags:
      - Users
    operationId: getAllUsers
    security:
      - bearerAuth: []
    responses:
      200:
        description: A list of all users.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/User'
      401:
        description: Unauthorized.
```

```yaml
      403:
        description: Forbidden, only admins can view all users.
      500:
        description: Internal server error.
/users/me/favorites:
  get:
    summary: Get current user's favorite cultural sites
    tags:
      - Users
    operationId: getFavoriteSites
    security:
      - bearerAuth: []
    responses:
      200:
        description: A list of favorite cultural sites.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/CulturalSite'
      401:
        description: Unauthorized.
      404:
        description: can't find the user.
      500:
        description: Internal server error.
/users/me/favorites/{siteId}:
  post:
    summary: Add a cultural site to current user's favorites
    tags:
      - Users
    operationId: addFavoriteSite
    security:
```

```
        - bearerAuth: []
      parameters:
        - in: path
          name: siteId
          schema:
            type: string
          required: true
          description: Unique ID of the cultural site to add to
favorites.
      responses:
        200:
          description: Cultural site added to favorites.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        400:
          description: Cultural site already in favorites.
        401:
          description: Unauthorized.
        404:
          description: Cultural site not found.
        500:
          description: Internal server error.
    delete:
      summary: Remove a cultural site from current user's favorites
      tags:
        - Users
      operationId: removeFavoriteSite
      security:
        - bearerAuth: []
      parameters:
        - in: path
          name: siteId
```

```
          schema:
            type: string
          required: true
          description: Unique ID of the cultural site to remove from
favorites.
      responses:
        200:
          description: Cultural site removed from favorites.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
        400:
          description: Cultural site not in favorites.
        401:
          description: Unauthorized.
        404:
          description: Cultural site not found.
        500:
          description: Internal server error.
  /users/me/reviews:
    get:
      summary: Get reviews submitted by the current user
      tags:
        - Users
      operationId: getMyReviews
      security:
        - bearerAuth: []
      responses:
        200:
          description: A list of reviews submitted by the user.
          content:
            application/json:
              schema:
```

```
                type: array
                items:
                  $ref: '#/components/schemas/Review'
        401:
          description: Unauthorized.
        500:
          description: Internal server error.
```